

Structures de Données et Algorithmes Avancés

Lab S01 — ADTs et Interfaces Java

Implémenter `ArrayDeque<T>` et le mettre à l'épreuve

Master 1 — 2025–2026

Dr. El Hadji Bassirou TOURÉ
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Avant de commencer

Prérequis : TP-1 complété (Docker, Gradle, JUnit opérationnels) et CM S01 lu.

Organisation : ce document comporte deux parties. *Partie I* — vous implémentez et vous observez. *Partie II* — vous établissez par le calcul ce que vous venez d'observer.

Ce Lab n'est pas noté. Il est formatif : son rôle est de rendre exécutable ce que le CM a énoncé.

Livable : le dépôt Git du projet `lab-s01` et le rapport individuel décrit en fin de document.

Ce que vous saurez faire à la fin de ce Lab

1. Créer un projet Gradle et y déclarer une interface générique.
2. Implémenter `ArrayDeque<T>` sur un tableau circulaire : les quatre opérations d'extrémité et le redimensionnement.
3. Écrire un test JUnit qui *force* le rebouclage du tableau, et un autre qui force le redimensionnement.
4. Lire un programme d'observation et y reconnaître le déplacement des indices `tete` et `fin`.
5. Restreindre un deque pour en faire une pile, et évaluer une expression postfixe.
6. Calculer à la main l'état d'un tableau circulaire après une séquence d'opérations.

Table des matières

I	Pratique : implémentation et observation	2
1	Environnement de travail	2
2	Étape 1 — Poser le contrat	2
3	Étape 2 — Le squelette et les champs	3

4	Étape 3 — Les quatre opérations d'extrémité	3
5	Étape 4 — Le redimensionnement	4
6	Étape 5 — Les tests	5
7	Étape 6 — Observer	6
8	Étape 7 — Une pile, et un évaluateur postfixe	7
9	Enregistrer le travail	8
II Théorie : exercices d'application		9
10	Exercice 1 — Arithmétique de l'anneau	9
11	Exercice 2 — Le modulo négatif	10
12	Exercice 3 — Trace d'une pile	10
13	Exercice 4 — Trace postfixe	10
14	Exercice 5 — Contrat et coût	11
15	Exercice 6 — Question subsidiaire	11
	Pour aller plus loin (optionnel)	12
	Aide-mémoire	12

Première partie

Pratique : implémentation et observation

1 Environnement de travail

Le travail se fait dans le conteneur installé au TP-1 : vous éditez les fichiers avec votre éditeur habituel sur votre machine, vous compilez et testez dans le conteneur.

Listing 1 – Rappel : lancer le conteneur depuis le dossier de travail

```
# Windows (Git Bash)
MSYS_NO_PATHCONV=1 docker run -it -v "$(pwd):/workspace"
    elbachirtoure/dsa-java-m1:v1.1

# macOS / Linux
docker run -it -v "$(pwd):/workspace" elbachirtoure/dsa-java-m1:v1.1
```

Listing 2 – Créer le projet

```
cd /workspace
new-project lab-s01
cd lab-s01
gradle build --no-daemon
```

Deux règles héritées du TP-1

La classe principale doit s'appeler `Main` — c'est ce que `new-project` inscrit dans `build.gradle`. Et toutes les commandes Gradle prennent `-no-daemon` : sans cette option, le démon Gradle échoue sur les volumes montés.

2 Étape 1 — Poser le contrat

Ce qu'on matérialise du CM

*L'ADT est un contrat, et en Java un contrat s'écrit **interface**. Rien dans ce fichier ne dit comment les éléments seront rangés : c'est précisément ce qui permettra d'en écrire plusieurs implémentations.*

Créez `src/main/java/Deque.java` :

Listing 3 – `Deque.java` — le contrat, fourni

```
1 public interface Deque<T> {
2     void addFirst(T element);
3     void addLast(T element);
4     T removeFirst();
5     T removeLast();
6     T peekFirst();
7     T peekLast();
8     int size();
9     boolean isEmpty();
10 }
```

3 Étape 2 — Le squelette et les champs

Ce qu'on matérialise du CM

Le tableau circulaire ne déplace jamais les éléments : il déplace les indices. Trois champs suffisent — le tableau, l'indice *tete* du premier élément, et le nombre d'éléments *taille*. L'indice de la prochaine case libre en fin se calcule par $(tete + taille) \bmod m$.

Créez `src/main/java/ArrayDeque.java` :

Listing 4 – `ArrayDeque.java` — champs et utilitaires

```

1 public class ArrayDeque<T> implements Deque<T> {
2
3     private Object[] donnees;
4     private int tete;           // indice du premier élément
5     private int taille;        // nombre d'éléments présents
6
7     public ArrayDeque() {
8         this.donnees = new Object[8];
9         this.tete = 0;
10        this.taille = 0;
11    }
12
13    public int size()           { return taille; }
14    public boolean isEmpty()    { return taille == 0; }
15    public int capacite()       { return donnees.length; } // utile
16                                pour observer
17    public int indiceTete()     { return tete; }           // utile
18                                pour observer
19
20    /** Avance ou recule un indice sur l'anneau. */
21    private int suivant(int i) { return (i + 1) % donnees.length; }
22    private int precedent(int i) { return (i - 1 + donnees.length) %
23        donnees.length; }
24 }
```

Le point sur lequel presque tout le monde trébuche

Dans `precedent`, le terme `+ donnees.length` n'est pas décoratif. En Java, $(0 - 1) \% 8$ vaut -1 , pas 7 : l'opérateur `%` renvoie un reste dont le signe suit le dividende. Sans ce correctif, le premier `addFirst` sur un deque vide lève une `ArrayIndexOutOfBoundsException`.

4 Étape 3 — Les quatre opérations d'extrémité

Ce qu'on matérialise du CM

Les quatre opérations écrivent ou lisent une seule case et bougent un seul indice. Aucune boucle : c'est la raison pour laquelle le deque est la structure de choix quand on travaille aux deux bouts.

Listing 5 – Les quatre opérations

```

1  public void addLast(T element) {
2      if (taille == donnees.length) agrandir();
3      donnees[(tete + taille) % donnees.length] = element;
4      taille++;
5  }
6
7  public void addFirst(T element) {
8      if (taille == donnees.length) agrandir();
9      tete = precedent(tete);
10     donnees[tete] = element;
11     taille++;
12 }
13
14 @SuppressWarnings("unchecked")
15 public T removeFirst() {
16     if (isEmpty()) throw new
17         java.util.NoSuchElementException("deque vide");
18     T premier = (T) donnees[tete];
19     donnees[tete] = null;
20     tete = suivant(tete);
21     taille--;
22     return premier;
23 }
24
25 @SuppressWarnings("unchecked")
26 public T removeLast() {
27     if (isEmpty()) throw new
28         java.util.NoSuchElementException("deque vide");
29     int dernier = (tete + taille - 1) % donnees.length;
30     T valeur = (T) donnees[dernier];
31     donnees[dernier] = null;
32     taille--;
33     return valeur;
34 }

```

Vérification 1 — à compléter

Écrivez `peekFirst()` et `peekLast()` sur le modèle ci-dessus. Ces deux méthodes ne modifient ni `tete` ni `taille`.

Quelle expression donne l'indice du dernier élément ?

5 Étape 4 — Le redimensionnement

Ce qu'on matérialise du CM

La capacité d'un tableau est finie. Quand le deque est plein, on alloue un tableau deux fois plus grand et on y recopie les éléments dans l'ordre logique, en repartant de `tete` : le nouveau tableau n'est plus rebouclé, et `tete` redevient 0.

Listing 6 – Agrandir en remettant l'anneau à plat

```

1  private void agrandir() {
2      Object[] neuf = new Object[donnees.length * 2];
3      for (int k = 0; k < taille; k++) {

```

```

4         neuf[k] = donnees[(tete + k) % donnees.length]; // ordre
               logique
5     }
6     donnees = neuf;
7     tete = 0;
8 }

```

L'erreur à ne pas commettre

Recopier avec `System.arraycopy(donnees, 0, neuf, 0, taille)` paraît plus rapide, mais copie les cases dans l'ordre *physique*. Si le deque a rebouclé, l'ordre logique est détruit et les tests d'ordre échouent sans que rien ne plante.

6 Étape 5 — Les tests

Ce qu'on matérialise du CM

Un test vérifie le contrat, pas l'implémentation : il n'a pas à connaître `tete`. Mais deux phénomènes internes doivent être provoqués volontairement, car ils ne surviennent pas dans un usage naïf : le rebouclage et le redimensionnement.

Créez `src/test/java/ArrayDequeTest.java` :

Listing 7 – Tests JUnit fournis

```

1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class ArrayDequeTest {
5
6     @Test
7     void ordreFifoEtLifo() {
8         Deque<String> d = new ArrayDeque<>();
9         d.addLast("Fatou");
10        d.addLast("Moussa");
11        assertEquals("Fatou", d.removeFirst()); // comportement de
               file
12        d.addFirst("Awa");
13        assertEquals("Awa", d.removeFirst()); // comportement de
               pile
14        assertEquals(1, d.size());
15    }
16
17    @Test
18    void testForcerLeRebouclage() {
19        // capacité initiale 8 : on remplit, on vide par devant, on
               remplit encore
20        Deque<Integer> d = new ArrayDeque<>();
21        for (int i = 0; i < 6; i++) d.addLast(i);
22        for (int i = 0; i < 4; i++) d.removeFirst(); // tete avance
               à 4
23        for (int i = 100; i < 104; i++) d.addLast(i); // la fin
               repasse par 0
24        assertEquals(6, d.size());
25        assertEquals(4, d.removeFirst());
26        assertEquals(103, d.removeLast());

```

```

27     }
28
29     @Test
30     void testForcerLeRedimensionnement() {
31         Deque<Integer> d = new ArrayDeque<>();
32         for (int i = 0; i < 100; i++) d.addFirst(i);    // dépasse
33         // largement 8
34         assertEquals(100, d.size());
35         assertEquals(99, d.removeFirst());            // ordre
36         // préservé
37         assertEquals(0, d.removeLast());
38     }
39
40     @Test
41     void videLeveUneException() {
42         Deque<String> d = new ArrayDeque<>();
43         assertThrows(java.util.NoSuchElementException.class,
44             d::removeFirst);
45     }
46 }

```

Listing 8 – Lancer les tests

```
gradle test --no-daemon
```

Vérification 2 — à compléter

Ajoutez deux tests de votre cru :

1. un test qui alterne `addFirst` et `addLast` puis vérifie l'ordre complet de sortie ;
2. un test qui vide entièrement le deque puis le remplit à nouveau.

Lequel des deux échoue si l'on oublie le `+ donnees.length` de `precedent` ?

7 Étape 6 — Observer

Ce qu'on matérialise du CM

Le CM affirme que les éléments ne bougent jamais et que ce sont les indices qui se déplacent. Ce programme le rend visible : après chaque opération, il affiche *tete*, l'indice de *fin*, la taille et la capacité, et signale les redimensionnements.

Listing 9 – Main.java — programme d'observation

```

1 public class Main {
2
3     static <T> void trace(String operation, ArrayDeque<T> d) {
4         int fin = (d.indiceTete() + d.size()) % d.capacite();
5         System.out.printf("%-16s tete=%2d fin=%2d taille=%2d
6                             capacite=%2d\n",
7                             operation, d.indiceTete(), fin, d.size(),
8                             d.capacite());
9     }
10
11     public static void main(String[] args) {

```

```

10     ArrayDeque<String> d = new ArrayDeque<>();
11     System.out.println("--- on remplit par la fin ---");
12     for (String s : new String[]{"A", "B", "C"}) { d.addLast(s);
13         trace("addLast(" + s + ")", d); }
14
15     System.out.println("--- on remplit par le debut : tete recule
16         ---");
17     for (String s : new String[]{"D", "E"}) { d.addFirst(s);
18         trace("addFirst(" + s + ")", d); }
19
20     System.out.println("--- on retire aux deux bouts ---");
21     d.removeLast(); trace("removeLast()", d);
22     d.addFirst("F"); trace("addFirst(F)", d);
23     d.removeFirst(); trace("removeFirst()", d);
24
25     System.out.println("--- on depasse la capacite ---");
26     for (int i = 0; i < 12; i++) { d.addLast("x" + i);
27         trace("addLast(x" + i + ")", d); }
28 }
29 }

```

Listing 10 – Exécuter

```
gradle run --no-daemon
```

Vérification 3 — notez vos observations

1. Après le premier `addFirst`, que vaut `tete`? _____ Pourquoi cette valeur? _____
2. À quelle opération la capacité change-t-elle pour la première fois? _____
3. Juste après un changement de capacité, que vaut `tete`? _____ Expliquez : _____
4. Relevez une ligne où `fin` est *inférieur* à `tete`. Que cela signifie-t-il? _____

8 Étape 7 — Une pile, et un évaluateur postfixe

Ce qu'on matérialise du CM

Le deque subsume la pile : en n'utilisant que `addLast` et `removeLast`, on obtient exactement le comportement dernier-arrivé-premier-sorti. On n'écrit donc pas une nouvelle structure — on restreint celle qu'on a.

Listing 11 – PostfixEvaluator.java

```

1 public class PostfixEvaluator {
2
3     public static int evaluer(String expression) {
4         ArrayDeque<Integer> pile = new ArrayDeque<>();
5         for (String jeton : expression.trim().split("\\s+")) {
6             if (estOperateur(jeton)) {
7                 if (pile.size() < 2) throw new
8                     IllegalArgumentException("operandes manquants");
9             }
10        }
11    }
12 }

```

```

8         int b = pile.removeLast();           // l'ordre compte
9         int a = pile.removeLast();
10        pile.addLast(appliquer(jeton, a, b));
11    } else {
12        pile.addLast(Integer.parseInt(jeton));
13    }
14    }
15    int resultat = pile.removeLast();
16    if (!pile.isEmpty()) throw new
        IllegalArgumentException("expression mal formee");
17    return resultat;
18    }
19
20    private static boolean estOperateur(String j) {
21        return j.equals("+") || j.equals("-") || j.equals("*") ||
            j.equals("/");
22    }
23
24    private static int appliquer(String op, int a, int b) {
25        switch (op) {
26            case "+": return a + b;
27            case "-": return a - b;
28            case "*": return a * b;
29            case "/": return a / b;
30            default: throw new IllegalArgumentException("operateur
                inconnu : " + op);
31        }
32    }
33    }

```

Vérification 4 — à compléter

Écrivez trois tests JUnit pour PostfixEvaluator :

1. "6 2 / 3 4 * +" doit valoir _____
2. une expression contenant une soustraction, pour détecter une inversion des opérands;
3. une expression mal formée, qui doit lever une exception.

9 Enregistrer le travail

Listing 12 – Git

```

cd /workspace/lab-s01
git init
git add .
git commit -m "Lab S01 : ArrayDeque, tests, evaluateur postfixe"

```

Deuxième partie

Théorie : exercices d'application

Comment aborder cette partie

Ces exercices ne sont pas un contrôle et ne sont pas notés. Ils reprennent exactement les situations que vous venez de programmer : le but est que vous retrouviez par le calcul ce que le programme d'observation vous a montré à l'écran. Remplissez directement sur ce document.

10 Exercice 1 — Arithmétique de l'anneau

Un `ArrayDeque` de capacité $m = 8$, initialement vide, avec `tete = 0`. Complétez le tableau après chaque opération. Les cases du tableau interne sont numérotées de 0 à 7.

Opération	tete	taille	case écrite ou libérée
<code>addLast(A)</code>			
<code>addLast(B)</code>			
<code>addLast(C)</code>			
<code>addFirst(D)</code>			
<code>addFirst(E)</code>			
<code>removeLast()</code>			
<code>addFirst(F)</code>			
<code>removeFirst()</code>			

Aide : la case écrite par `addLast` est $(tete + taille) \bmod 8$; `addFirst` écrit dans $(tete - 1 + 8) \bmod 8$.

(a) Quelles valeurs ont été retirées, et dans cet ordre ? _____

(b) Dessinez l'état final des huit cases :

0	1	2	3	4	5	6	7
_____	_____	_____	_____	_____	_____	_____	_____

(c) À la fin, `tete` vaut-il un indice inférieur ou supérieur à celui de la case de fin ? Qu'est-ce que cela indique sur l'état du tableau ? _____

11 Exercice 2 — Le modulo négatif

(a) En Java, donnez la valeur de chaque expression :

Expression	Valeur
$(3 - 1) \% 8$	
$(0 - 1) \% 8$	
$(0 - 1 + 8) \% 8$	
$(7 + 1) \% 8$	

(b) Un étudiant écrit `tete = (tete - 1) % donnees.length;` puis appelle `addFirst` sur un deque vide. Que se passe-t-il exactement, et à quelle ligne ? _____

(c) Ce bug est-il détecté par un test qui n'utilise que `addLast` et `removeFirst` ? Justifiez. _____

12 Exercice 3 — Trace d'une pile

Déroulez la vérification de parenthésage sur `{ a ( b [ c ] ) d }` en notant le contenu de la pile après chaque symbole significatif (on ignore les lettres).

Symbole lu	Action	Pile après
------------	--------	------------

(a) L'expression est-elle équilibrée ? _____

(b) Modifiez un seul symbole de l'expression pour qu'elle soit rejetée *par le test final* (pile non vide à la fin), et non par une correspondance fautive : _____

13 Exercice 4 — Trace postfixe

Évaluez `6 2 / 3 4 * + 5 -` en notant la pile après chaque jeton.

6	2	/	3	4	*	+	5	-
---	---	---	---	---	---	---	---	---

- (a) Résultat : _____
- (b) Hauteur maximale atteinte par la pile : _____
- (c) Si l'on inverse les deux dépilements dans le code, quel résultat obtient-on pour cette expression ? _____

14 Exercice 5 — Contrat et coût

Pour chaque opération, indiquez si le travail effectué est indépendant du nombre n d'éléments présents, ou s'il croît avec n .

Opération	Tableau contigu	Nœuds chaînés
<code>get(i)</code>		
<code>ajouter(x)</code> en fin		
<code>ajouter(0, x)</code> en tête		
<code>supprimer(0)</code>		
<code>taille()</code>		

- (a) Un collègue implémente une file avec une `ArrayList` : `enqueue` $\rightarrow$ `add(x)`, `dequeue` $\rightarrow$ `remove(0)`. Le contrat de l'ADT Queue est-il respecté ? _____
- (b) Un test JUnit portant uniquement sur le contrat peut-il révéler le problème ? _____
- (c) Comment le mettriez-vous en évidence expérimentalement ? _____

15 Exercice 6 — Question subsidiaire

On voudrait ajouter au contrat `Deque<T>` une opération `get(int i)` renvoyant le i -ième élément depuis la tête.

- (a) Écrivez l'expression de l'indice interne correspondant : _____
- (b) Cette opération a-t-elle sa place dans le contrat d'un deque ? Argumentez en une ou deux phrases. _____

Pour aller plus loin (optionnel)

- Écrire `LinkedDeque<T>`, seconde implémentation du même contrat, à l'aide de nœuds doublement chaînés — et faire passer *les mêmes tests* sans y changer une ligne. C'est la démonstration expérimentale du principe de la séance.
- Ajouter au deque une capacité maximale avec éviction automatique du plus ancien : vous obtenez un historique borné.
- Étendre l'évaluateur postfixe à la puissance et aux nombres négatifs.

Aide-mémoire

Question	Réponse
Indice de la prochaine case en fin	$(\text{tete} + \text{taille}) \bmod m$
Indice du dernier élément	$(\text{tete} + \text{taille} - 1) \bmod m$
Reculer un indice	$(i - 1 + m) \bmod m$
Avancer un indice	$(i + 1) \bmod m$
Deque vide	<code>taille == 0</code> (jamais <code>tete == fin</code>)
Après <code>agrandir()</code>	<code>tete</code> redevient 0
Lancer le conteneur	<code>docker run -it -v "\$(pwd):/workspace" ...:v1.1</code>
Créer un projet	<code>new-project lab-s01</code>
Compiler	<code>gradle build --no-daemon</code>
Tester	<code>gradle test --no-daemon</code>
Exécuter	<code>gradle run --no-daemon</code>

Rapport individuel — obligatoire

Chaque étudiant rédige un rapport de **1 à 2 pages** :

1. **Ce que j'ai fait** : classes implémentées, tests écrits, ce qu'affiche `Main`.
2. **Les observations clés** : qu'est-ce que le programme d'observation m'a montré que je n'avais pas bien compris du CM ?
3. **Les difficultés** : erreurs de compilation, bugs, blocages sur les exercices théoriques.
4. **Ce que j'ai appris** : le concept le plus important retenu de cette séance.

Format : PDF. **Nom** : `rapport-lab-S01-PRENOM-NOM.pdf`